Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040079
Nama	M Ali Fauzi JR
Kelas	A
Link Github Repository	https://github.com/awithliee/Praktikum_Web_243040079.git

Bagian 1. Screenshot Semua Kode

Berikut tangkapan seluruh kode sumber aplikasi Catatan Keuangan Mini pada folder src/ proyek praktikum-react-modul2-keuangan: main.jsx, App.jsx, Kartu.jsx, RingkasanSaldo.jsx, FormTransaksi.jsx, ItemTransaksi.jsx, formatRupiah.jsx, index.css, dan App.css.

praktikum-react-modul2-keuangan > src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Gambar 1. Kode src/main.jsx (entry point aplikasi)

App.jsx M

praktikum-react-modul2-keuangan > src > App.jsx > ...

```jsx
// src/App.jsx
import { useState, useEffect } from 'react';
import './App.css';
import Kartu from './Kartu';
import RingkasanSaldo from './RingkasanSaldo';
import FormTransaksi from './FormTransaksi';
import ItemTransaksi from './ItemTransaksi';

const KUNCI_STORAGE = 'catatan-keuangan';

function App() {
  // State utama: baca dari localStorage saat pertama kali tampil
  const [daftarTransaksi, setDaftarTransaksi] = useState(() => {
    try {
      const dataTersimpan = localStorage.getItem(KUNCI_STORAGE);
      return dataTersimpan ? JSON.parse(dataTersimpan) : [];
    } catch {
      return []; // jika isi localStorage rusak, mulai dari kosong
    }
  });

  // useEffect: simpan ke localStorage setiap daftarTransaksi berubah
  useEffect(() => {
    localStorage.setItem(KUNCI_STORAGE, JSON.stringify(daftarTransaksi));
  }, [daftarTransaksi]);

  // Menambah transaksi: array BARU berisi transaksi baru + transaksi lama
  const tambahTransaksi = (dataBaru) => {
    const transaksiBaru = { id: Date.now(), ...dataBaru };
    setDaftarTransaksi([transaksiBaru, ...daftarTransaksi]);
  };

  // Menghapus transaksi: ambil semua KECUALI yang id-nya sama
  const hapusTransaksi = (id) => {
    setDaftarTransaksi(daftarTransaksi.filter((transaksi) => transaksi.id !== id));
  };

  // Nilai turunan: dihitung dari state, tidak disimpan sebagai state baru
  const totalMasuk = daftarTransaksi
    .filter((transaksi) => transaksi.jenis === 'pemasukan')
    .reduce((jumlah, transaksi) => jumlah + transaksi.nominal, 0);

  const totalKeluar = daftarTransaksi
    .filter((transaksi) => transaksi.jenis === 'pengeluaran')
    .reduce((jumlah, transaksi) => jumlah + transaksi.nominal, 0);
```

Gambar 2. Kode src/App.jsx (komponen induk dan state) - bagian 1 dari 2, baris 1-45

```jsx
  const saldo = totalMasuk - totalKeluar;

  return (
    <div className="wadah">
      <h1 className="judul">Catatan Keuangan Mini</h1>
      <p className="subjudul">Laboratorium Informatika - Modul 2: React Lanjutan</p>

      <Kartu judul="Ringkasan">
        <RingkasanSaldo saldo={saldo} totalMasuk={totalMasuk} totalKeluar={totalKeluar} />
      </Kartu>

      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambahTransaksi} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {/* Conditional rendering: pesan khusus jika belum ada transaksi */}
        {daftarTransaksi.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Tambahkan transaksi pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftarTransaksi.map((transaksi) => (
              <ItemTransaksi
                key={transaksi.id}
                transaksi={transaksi}
                onHapus={hapusTransaksi}
              />
            ))}
          </ul>
        )}
      </Kartu>
    </div>
  );
}

export default App;
```

Gambar 3. Kode src/App.jsx - bagian 2 dari 2, baris 47-82

App.jsx M Kartu.jsx

praktikum-react-modul2-keuangan > src > Kartu.jsx > ...

```jsx
// src/Kartu.jsx
function Kartu({ judul, children }) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

Gambar 4. Kode src/Kartu.jsx (komponen pembungkus)

App.jsx M | Kartu.jsx M | RingkasanSaldo.jsx

praktikum-react-modul2-keuangan > src > RingkasanSaldo.jsx > ...

```jsx
// src/RingkasanSaldo.jsx
import { formatRupiah } from './formatRupiah';

function RingkasanSaldo({ saldo, totalMasuk, totalKeluar }) {
  return (
    <div>
      <p className="saldo-label">Saldo saat ini</p>
      {/* className dinamis: hijau jika saldo >= 0, merah jika minus */}
      <p className={saldo >= 0 ? 'saldo-nilai pemasukan' : 'saldo-nilai pengeluaran'}>
        {formatRupiah(saldo)}
      </p>

      <div className="ringkasan-baris">
        <div className="ringkasan-kotak kotak-masuk">
          <small>Total Pemasukan</small>
          <strong className="pemasukan">{formatRupiah(totalMasuk)}</strong>
        </div>
        <div className="ringkasan-kotak kotak-keluar">
          <small>Total Pengeluaran</small>
          <strong className="pengeluaran">{formatRupiah(totalKeluar)}</strong>
        </div>
      </div>
    </div>
  );
}

export default RingkasanSaldo;
```

Gambar 5. Kode src/RingkasanSaldo.jsx (ringkasan saldo)

App.jsx M | Kartu.jsx M | RingkasanSaldo.jsx M | FormTransaksi.jsx M

praktikum-react-modul2-keuangan > src > FormTransaksi.jsx > ...

```jsx
// src/FormTransaksi.jsx
import { useState } from 'react';

function FormTransaksi({ onTambah }) {
  const [keterangan, setKeterangan] = useState('');
  const [nominal, setNominal] = useState(''); // nilai input selalu berupa teks
  const [jenis, setJenis] = useState('pemasukan');

  // Validasi
  const keteranganTerlaluPanjang = keterangan.length > 40;
  const nominalTidakValid = nominal !== '' && Number(nominal) <= 0;
  const tidakValid =
    keterangan.trim() === '' ||
    keteranganTerlaluPanjang ||
    nominal === '' ||
    Number(nominal) <= 0;

  const tanganiKirim = (event) => {
    event.preventDefault(); // cegah reload halaman
    if (tidakValid) return;

    // Number(nominal) agar "5000" + "2000" tidak menjadi "50002000"
    onTambah({
      keterangan: keterangan.trim(),
      nominal: Number(nominal),
      jenis: jenis,
    });

    // kosongkan kembali formulir
    setKeterangan('');
    setNominal('');
    setJenis('pemasukan');
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="form-grup">
        <label htmlFor="keterangan">Keterangan</label>
        <input
          id="keterangan"
          className="input"
          type="text"
          placeholder="Contoh: Uang saku, makan siang"
          value={keterangan}
          onChange={(event) => setKeterangan(event.target.value)}
        />
        {keteranganTerlaluPanjang && (
          <p className="galat">Keterangan maksimal 40 karakter.</p>
        )}
      </div>
```

Gambar 6. Kode src/FormTransaksi.jsx (formulir terkendali) - bagian 1 dari 2, baris 1-50



```
function FormTransaksi({ onTambah }) {

      <div className="baris form-grup">
        <div>
          <label htmlFor="nominal">Nominal (Rp)</label>
          <input
            id="nominal"
            className="input"
            type="number"
            placeholder="0"
            value={nominal}
            onChange={(event) => setNominal(event.target.value)}
          />
        </div>

        <div>
          <label htmlFor="jenis">Jenis</label>
          <select
            id="jenis"
            className="input"
            value={jenis}
            onChange={(event) => setJenis(event.target.value)}
          >
            <option value="pemasukan">Pemasukan</option>
            <option value="pengeluaran">Pengeluaran</option>
          </select>
        </div>
      </div>

      {nominalTidakValid && <p className="galat">Nominal harus lebih dari 0.</p>}

      <button className="tombol" type="submit" disabled={tidakValid}>
        Tambah Transaksi
      </button>
    </form>
  );
}

export default FormTransaksi;
```

Gambar 7. Kode src/FormTransaksi.jsx - bagian 2 dari 2, baris 52-88

App.jsx M | Kartu.jsx M | RingkasanSaldo.jsx M | FormTransaksi.jsx M | ItemTransaksi.jsx

praktikum-react-modul2-keuangan > src > ItemTransaksi.jsx > ...

```jsx
// src/ItemTransaksi.jsx
// Satu baris transaksi + tombol hapus
import { formatRupiah } from "./formatRupiah";

function ItemTransaksi({ transaksi, onHapus }) {
  const adalahPemasukan = transaksi.jenis === "pemasukan";

  return (
    <li className="item">
      <div className="item-info">
        <strong>{transaksi.keterangan}</strong>
        <small>{adalahPemasukan ? "Pemasukan" : "Pengeluaran"}</small>
      </div>

      {/* Hijau untuk pemasukan, merah untuk pengeluaran */}
      <span className={`item-nominal ${transaksi.jenis}`}>
        {adalahPemasukan ? "+" : "-"} {formatRupiah(transaksi.nominal)}
      </span>

      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

Gambar 8. Kode src/ItemTransaksi.jsx (satu baris transaksi)

App.jsx M | Kartu.jsx M | RingkasanSaldo.jsx M | FormTransaksi.jsx M | formatRupiah.jsx M

praktikum-react-modul2-keuangan > src > formatRupiah.jsx > formatRupiah

```jsx
// src/formatRupiah.jsx
// Mengubah angka menjadi teks rupiah, contoh: 15000 -> "Rp 15.000"
export function formatRupiah(angka) {
  const tanda = angka < 0 ? '-' : '';
  return `${tanda}Rp ${Math.abs(angka).toLocaleString('id-ID')}`;
}
```

Gambar 9. Kode src/formatRupiah.jsx (fungsi format rupiah)



```
@import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap');

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: 'Plus Jakarta Sans', system-ui, 'Segoe UI', sans-serif;
  font-size: 15px;
  color: #1b2a41;
  background-color: #eef2f8;
}
```

Gambar 10. Kode src/index.css (gaya global)

App.jsx M | Kartu.jsx M | RingkasanSaldo.jsx M | FormTransaksi.jsx M | formatRupiah.jsx M | App.css M

praktikum-react-modul2-keuangan > src > App.css > .wadah

```css
.wadah { max-width: 440px; margin: 0 auto; padding: 28px 16px 56px; }

/* ===== Header ===== */
.judul { margin: 0; font-size: 19px; font-weight: 800; color: #0a2e6e; }
.subjudul { margin: 4px 0 22px; font-size: 13px; color: #6b7a90; }
.topbar { display: flex; align-items: center; gap: 12px; margin-bottom: 22px; }
.topbar .subjudul { margin: 2px 0 0; }
.logo, .wadah > .judul::before {
  width: 42px; height: 42px; border-radius: 13px;
  display: inline-flex; align-items: center; justify-content: center;
  background: linear-gradient(135deg, #0a2e6e, #1456c8);
  color: #ffffff; font-size: 20px; font-weight: 800;
}
.wadah > .judul::before { content: "M"; width: 38px; height: 38px; margin-right: 10px; vertical-align: middle; }

/* ===== Kartu putih ===== */
.kartu {
  background: #ffffff;
  border-radius: 20px;
  padding: 22px 20px;
  margin-bottom: 18px;
  box-shadow: 0 6px 20px rgba(10, 46, 110, 0.08);
}
.kartu h2 { margin: 0 0 16px; font-size: 16px; font-weight: 700; color: #0a2e6e; }

/* ===== Kartu saldo bergaya kartu bank ===== */
.kartu-bank, .kartu:has(.saldo-nilai) {
  position: relative;
  overflow: hidden;
  margin-bottom: 18px;
  padding: 22px 22px 20px;
  border-radius: 24px;
  color: #ffffff;
  background: linear-gradient(135deg, #0a2e6e 0%, #1456c8 58%, #2b8cff 100%);
  box-shadow: 0 14px 30px rgba(20, 86, 200, 0.38);
}
.kartu-bank::after, .kartu:has(.saldo-nilai)::after {
  content: "";
  position: absolute; top: -70px; right: -60px;
  width: 210px; height: 210px; border-radius: 50%;
  background: rgba(255, 255, 255, 0.09);
}
.kartu-bank::before {
  content: "";
  position: absolute; bottom: -90px; left: -50px;
  width: 200px; height: 200px; border-radius: 50%;
  background: rgba(255, 255, 255, 0.06);
}
.kartu-bank > *, .kartu:has(.saldo-nilai) > * { position: relative; z-index: 1; }
.kartu-bank-atas { display: flex; justify-content: space-between; align-items: center; margin-bottom: 24px; }
.bank-nama { font-size: 13px; font-weight: 800; letter-spacing: 3px; }
.chip, .kartu:has(.saldo-nilai) > h2::after {
  display: block; width: 40px; height: 30px; border-radius: 7px;
  background: linear-gradient(135deg, #f7e0a0, #c9a23f);
}
```

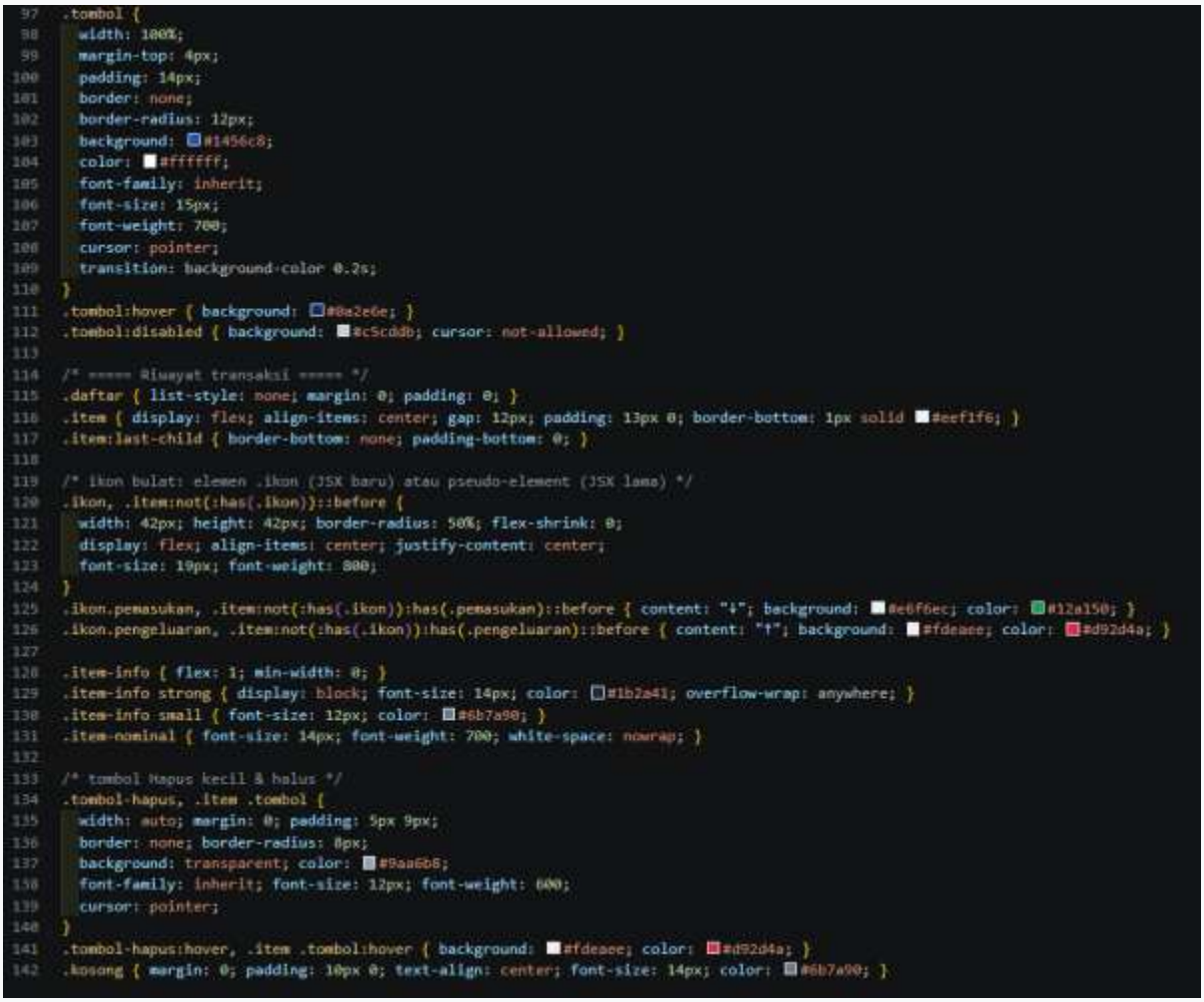
Gambar 11. Kode src/App.css (gaya komponen) - bagian 1 dari 3, baris 1-55

```
.kartu:has(.saldo-nilai) > h2 {
  display: flex; justify-content: space-between; align-items: center;
  margin-bottom: 24px;
  font-size: 13px; font-weight: 800; letter-spacing: 3px; text-transform: uppercase;
  color: #ffffff;
}
.saldo-label { margin: 0; font-size: 13px; color: rgba(255, 255, 255, 0.78); }
.saldo-nilai { margin: 4px 0 20px; font-size: 32px; font-weight: 800; letter-spacing: -0.5px; }
.ringkasan-baris { display: flex; gap: 10px; }
.ringkasan-kotak { flex: 1; padding: 11px 13px; border-radius: 14px; background: rgba(255, 255, 255, 0.15); }
.ringkasan-kotak small { display: block; margin-bottom: 3px; font-size: 12px; color: rgba(255, 255, 255, 0.82); }
.ringkasan-kotak strong { font-size: 15px; }

/* ===== Warna: hijau pemasukan, merah pengeluaran ===== */
.pemasukan { color: #12a150; }
.pengeluaran { color: #d92d4a; }
.kartu-bank .pemasukan, .kartu:has(.saldo-nilai) .pemasukan { color: #8cf5b8; }
.kartu-bank .pengeluaran, .kartu:has(.saldo-nilai) .pengeluaran { color: #ffb3c0; }
.kartu-bank .saldo-nilai, .kartu:has(.saldo-nilai) .saldo-nilai { color: #ffffff; }
.kartu-bank .saldo-nilai.pengeluaran, .kartu:has(.saldo-nilai) .saldo-nilai.pengeluaran { color: #ffb3c0; }

/* ===== Formulir ===== */
.form-grup { margin-bottom: 14px; }
.form-grup label { display: block; margin-bottom: 6px; font-size: 13px; font-weight: 600; color: #6b7a90; }
.baris { display: flex; gap: 10px; }
.baris > * { flex: 1; min-width: 0; }
.input {
  width: 100%;
  padding: 13px 14px;
  border: 1px solid #dde3ee;
  border-radius: 12px;
  background: #f5f7fb;
  color: #1b2a41;
  font-family: inherit;
  font-size: 15px;
}
.input::placeholder { color: #a3aec0; }
.input:focus { outline: 2px solid #1456c8; outline-offset: 1px; background: #ffffff; }
.galat { margin: 6px 0 0; font-size: 13px; color: #d92d4a; }
```

Gambar 12. Kode src/App.css - bagian 2 dari 3, baris 57-95



```
.tombol {
  width: 100%;
  margin-top: 4px;
  padding: 14px;
  border: none;
  border-radius: 12px;
  background: #1456c8;
  color: #ffffff;
  font-family: inherit;
  font-size: 15px;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 0.2s;
}
.tombol:hover { background: #0a2e6e; }
.tombol:disabled { background: #c5cddb; cursor: not-allowed; }

/* ===== Riwayat transaksi ===== */
.daftar { list-style: none; margin: 0; padding: 0; }
.item { display: flex; align-items: center; gap: 12px; padding: 13px 0; border-bottom: 1px solid #eef1f6; }
.item:last-child { border-bottom: none; padding-bottom: 0; }

/* ikon bulat: elemen .ikon (JSX baru) atau pseudo-element (JSX lama) */
.ikon, .item:not(:has(.ikon))::before {
  width: 42px; height: 42px; border-radius: 50%; flex-shrink: 0;
  display: flex; align-items: center; justify-content: center;
  font-size: 19px; font-weight: 800;
}
.ikon.pemasukan, .item:not(:has(.ikon)):has(.pemasukan)::before { content: "↓"; background: #e6f6ec; color: #12a150; }
.ikon.pengeluaran, .item:not(:has(.ikon)):has(.pengeluaran)::before { content: "↑"; background: #fdeaee; color: #d92d4a; }

.item-info { flex: 1; min-width: 0; }
.item-info strong { display: block; font-size: 14px; color: #1b2a41; overflow-wrap: anywhere; }
.item-info small { font-size: 12px; color: #6b7a90; }
.item-nominal { font-size: 14px; font-weight: 700; white-space: nowrap; }

/* tombol Hapus kecil & halus */
.tombol-hapus, .item .tombol {
  width: auto; margin: 0; padding: 5px 9px;
  border: none; border-radius: 8px;
  background: transparent; color: #9aa6b8;
  font-family: inherit; font-size: 12px; font-weight: 600;
  cursor: pointer;
}
.tombol-hapus:hover, .item .tombol:hover { background: #fdeaee; color: #d92d4a; }
.kosong { margin: 0; padding: 10px 0; text-align: center; font-size: 14px; color: #6b7a90; }
```

Gambar 13. Kode src/App.css - bagian 3 dari 3, baris 97-142

Bagian 2. Penjelasan Kode

Aplikasi Catatan Keuangan Mini dibangun dengan React dan Vite. Kode dipecah menjadi lima komponen (App, Kartu, RingkasanSaldo, FormTransaksi, ItemTransaksi), satu fungsi bantu (formatRupiah), dan dua berkas gaya, supaya setiap berkas hanya memiliki satu tanggung jawab. Penjelasan berikut menguraikan cara kerja sekaligus alasan di balik pilihan penulisan kodenya.

1. main.jsx. Berkas ini adalah titik masuk aplikasi. createRoot(...).render(...) memasang komponen <App /> ke elemen div dengan id root pada index.html, dibungkus <StrictMode> agar potensi masalah terdeteksi selama pengembangan. index.css diimpor di sini agar gaya global berlaku pada seluruh halaman.

2. App.jsx. State daftarTransaksi diletakkan di App karena dibutuhkan tiga bagian sekaligus: ringkasan (membaca total), formulir (menambah data), dan daftar (menampilkan dan menghapus data). Dengan menaruhnya di komponen induk terdekat yang menaungi ketiganya (lifting state up), data hanya memiliki satu sumber kebenaran sehingga tidak mungkin ada dua bagian yang menampilkan angka berbeda.

useState menerima sebuah fungsi (lazy initializer) sehingga pembacaan localStorage hanya terjadi sekali saat aplikasi pertama kali tampil, bukan pada setiap render. Blok try/catch dipakai agar aplikasi tidak berhenti total jika isi localStorage rusak. useEffect dengan dependency array [daftarTransaksi] dipakai karena menyimpan ke localStorage adalah sinkronisasi ke sistem di luar React. Efek hanya berjalan ketika daftar benar-benar berubah, dan data yang tersimpan selalu sama dengan state terbaru.

Fungsi tambahTransaksi membuat array baru dengan spread ([transaksiBaru, ...daftarTransaksi]) dan hapusTransaksi memakai .filter(). Keduanya tidak mengubah array lama (immutable), karena React hanya mendeteksi perubahan jika referensi state berbeda. Jika array diubah langsung dengan push() atau splice(), tampilan tidak diperbarui. Setiap transaksi diberi id dari Date.now() agar bisa dipakai sebagai key yang unik pada .map().

Nilai totalMasuk, totalKeluar, dan saldo dihitung dengan .filter() dan .reduce() langsung di badan komponen, tidak disimpan sebagai state baru. Alasannya, nilai itu selalu bisa diturunkan dari daftarTransaksi. Menyimpannya sebagai state justru membuka peluang datanya tidak sinkron. Pesan "Belum ada transaksi" ditampilkan dengan conditional rendering (operator ternary) yang memeriksa daftarTransaksi.length === 0.

3. Kartu.jsx. Komponen pembungkus yang menerima judul lewat props dan isinya lewat children. Alasannya, tampilan kartu dipakai berulang pada tiga bagian halaman. Dengan satu komponen, markup dan gayanya konsisten dan tidak perlu ditulis berulang.

4. RingkasanSaldo.jsx. Komponen presentasi murni: menerima saldo, totalMasuk, dan totalKeluar lewat props, tanpa state sendiri. Komponen seperti ini mudah dipahami dan dipakai ulang karena hasil tampilannya hanya ditentukan oleh props. Kelas pemasukan atau

pengeluaran dipilih secara dinamis berdasarkan saldo >= 0, sehingga saldo negatif otomatis berwarna merah.

5. FormTransaksi.jsx. Setiap input adalah controlled input: nilainya diambil dari state (value) dan diperbarui lewat onChange. Dengan cara ini React menjadi satu-satunya pemilik nilai input, sehingga isinya dapat divalidasi dan dikosongkan kembali setelah dikirim. Nilai input selalu berupa teks, bahkan pada type="number". Karena itu nominal diubah dengan Number(nominal) sebelum dikirim. Tanpa konversi, "5000" + "2000" akan menjadi "50002000", bukan 7000.

Validasi memastikan keterangan tidak kosong dan maksimal 40 karakter, serta nominal lebih dari 0. Selama tidak valid, tombol dinonaktifkan dan pesan galat ditampilkan. event.preventDefault() dipakai pada onSubmit agar halaman tidak dimuat ulang oleh perilaku bawaan formulir. Komponen ini tidak menyimpan daftar transaksi. Ia hanya melapor ke induk lewat function props onTambah, sehingga aliran data tetap satu arah dari induk ke anak.

6. ItemTransaksi.jsx. Menampilkan satu baris transaksi dan menerima transaksi serta onHapus lewat props. Tombol hapus ditulis onClick={() => onHapus(transaksi.id)} (arrow function). Jika ditulis onClick={onHapus(transaksi.id)}, fungsi akan langsung terpanggil saat render, bukan saat tombol diklik. Kelas item-nominal ditambah nilai transaksi.jenis sehingga pemasukan berwarna hijau dan pengeluaran berwarna merah sesuai ketentuan.

7. formatRupiah.js. Fungsi murni yang mengubah angka menjadi teks rupiah, misalnya 15000 menjadi "Rp 15.000". toLocaleString("id-ID") menghasilkan pemisah ribuan titik sesuai kebiasaan Indonesia, dan Math.abs dengan tanda terpisah memastikan angka negatif tampil sebagai -Rp 6.000. Fungsi ini dibuat di berkas sendiri karena dipakai oleh dua komponen sehingga tidak perlu ditulis dua kali.

8. index.css dan App.css. index.css memuat font dan pengaturan dasar halaman. App.css mengatur tampilan bergaya aplikasi mobile banking: lebar dibatasi 440 px agar seperti layar ponsel, kartu saldo berlatar biru bergradasi dengan chip emas, dan ikon panah bulat pada tiap transaksi. Warna hijau dan merah diatur lewat kelas .pemasukan dan .pengeluaran, serta tombol yang nonaktif diberi warna abu-abu melalui .tombol:disabled.

Dengan susunan ini seluruh ketentuan Tugas 3 terpenuhi: (a) formulir terkendali berisi keterangan, nominal, dan jenis; (b) daftar transaksi dengan tombol hapus; (c) saldo dihitung otomatis dari state; (d) pemasukan berwarna hijau dan pengeluaran berwarna merah; (e) pesan khusus saat belum ada transaksi; (f) data tersimpan di localStorage lewat useEffect; serta lima komponen terpisah.

Bagian 3. Screenshot Hasil Antarmuka

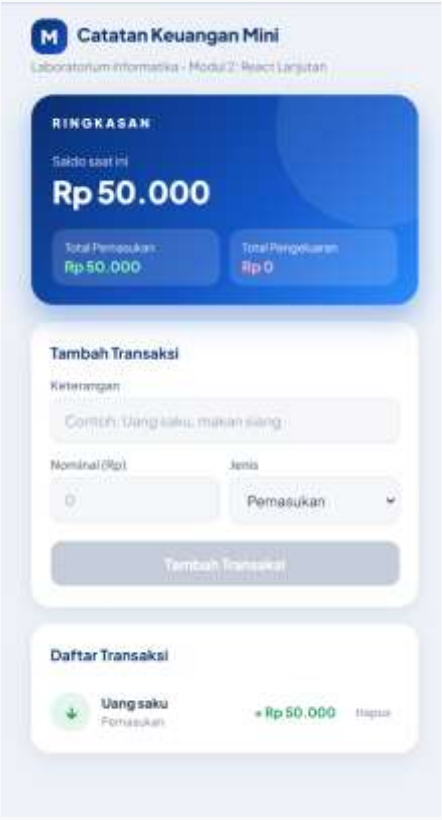
Berikut tangkapan layar untuk setiap skenario uji. Data uji dibuat berurutan agar angka pada tiap gambar saling berkesinambungan.



Gambar 14. Skenario 1 (kondisi awal, daftar kosong)



Gambar 15. Skenario 2 (validasi formulir)



Gambar 16. Skenario 3 (menambah pemasukan)



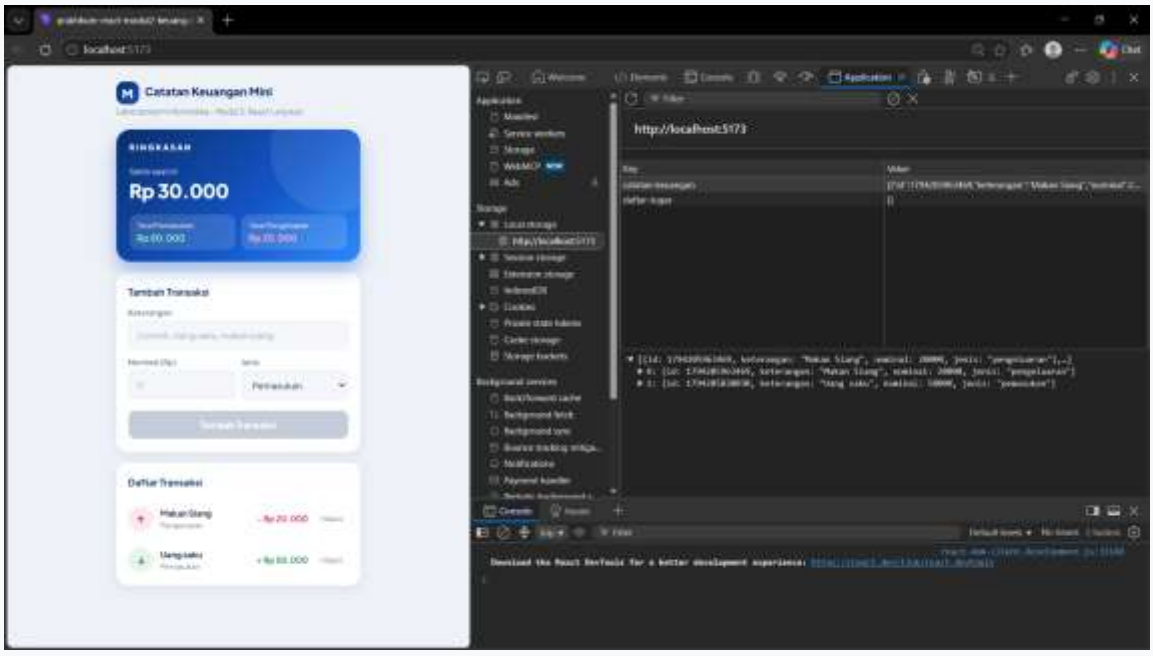
Gambar 17. Skenario 4 (menambah pengeluaran)



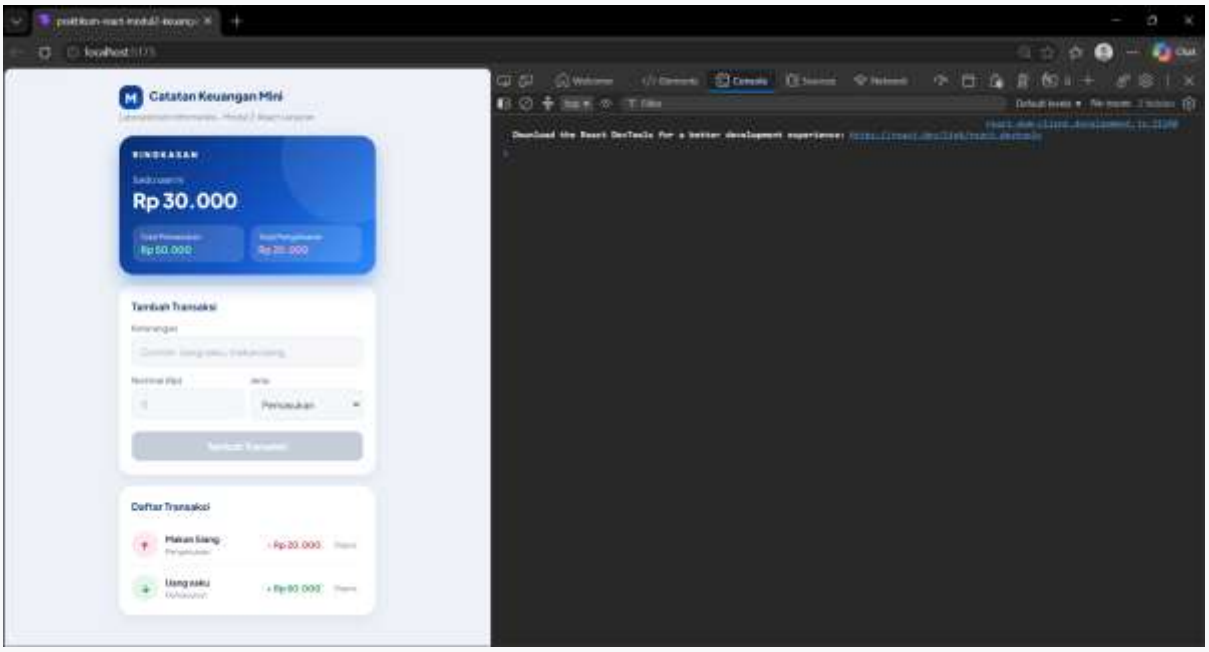
Gambar 18. Skenario 5 (saldo negatif)



Gambar 19. Skenario 6 (menghapus transaksi)



Gambar 20. Skenario 7 (data bertahan setelah refresh)



Gambar 21. Skenario 8 (konsol bersih)

Bagian 4. Penjelasan Hasil Antarmuka

Aplikasi dijalankan dengan perintah npm run dev dan dibuka pada peramban di alamat http://localhost:5173/. Hasil setiap skenario uji dijelaskan berikut ini.

Skenario 1: kondisi awal. Saldo menunjukkan Rp 0, total pemasukan dan pengeluaran juga Rp 0, dan bagian riwayat menampilkan pesan "Belum ada transaksi. Tambahkan transaksi pertamamu!". Pesan ini muncul karena conditional rendering memeriksa daftarTransaksi.length === 0. Tombol Tambah Transaksi berwarna abu-abu (nonaktif) karena formulir masih kosong.

Skenario 2: validasi. Ketika keterangan diisi "Test" dan nominal diisi 0, muncul pesan merah "Nominal harus lebih dari 0." dan tombol tetap nonaktif. Jika keterangan melebihi 40 karakter, muncul pesan "Keterangan maksimal 40 karakter.". Validasi ini mencegah data yang tidak bermakna masuk ke dalam daftar.

Skenario 3: menambah pemasukan. Setelah "Uang saku" senilai 5000 berjenis Pemasukan ditambahkan, transaksi tampil di riwayat dengan ikon panah hijau dan nominal "+ Rp 5.000" berwarna hijau. Saldo menjadi Rp 5.000. Isian formulir kosong kembali karena state input di-reset setelah data dikirim.

Skenario 4: menambah pengeluaran. Setelah "Makan siang" senilai 2000 berjenis Pengeluaran ditambahkan, transaksi tampil dengan ikon merah dan nominal "- Rp 2.000". Saldo menjadi Rp 3.000 (5.000 dikurangi 2.000), total pemasukan Rp 5.000, dan total pengeluaran Rp 2.000. Hasil yang benar ini menunjukkan konversi Number(nominal) bekerja. Tanpa konversi, angka akan digabung sebagai teks.

Skenario 5: saldo negatif. Setelah "Beli buku" senilai 9000 berjenis Pengeluaran ditambahkan, saldo menjadi -Rp 6.000 dan ditampilkan dengan warna merah muda, bukan putih seperti saldo positif. Perubahan warna ini berasal dari className dinamis yang memeriksa apakah saldo >= 0.

Skenario 6: menghapus transaksi. Ketika tombol Hapus pada "Beli buku" diklik, transaksi itu hilang dari daftar dan saldo kembali menjadi Rp 3.000. Saldo ikut berubah tanpa kode tambahan karena dihitung otomatis dari state (nilai turunan), bukan disimpan terpisah.

Skenario 7: data bertahan setelah refresh. Setelah halaman dimuat ulang dengan F5, seluruh transaksi dan saldo tetap tampil. Pada tab Application > Local Storage terlihat kunci catatan-keuangan berisi array JSON dari transaksi. Hal ini terjadi karena useEffect menyimpan setiap perubahan daftar, dan initializer useState membacanya kembali saat aplikasi dibuka.

Skenario 8: konsol. Konsol peramban tidak menampilkan error maupun peringatan, termasuk peringatan "unique key", karena setiap item daftar memiliki key unik berupa id.

Secara visual, aplikasi tampil seperti aplikasi mobile banking: kartu saldo biru bergradasi dengan chip emas di bagian atas, formulir berbentuk kartu putih, serta riwayat transaksi dengan ikon panah bulat (hijau untuk uang masuk, merah untuk uang keluar). Tampilan ini sesuai dengan kode pada App.css dan komponen yang telah dijelaskan.